of `m`, we can have a different number of choices for the split index.
(Bottom) We show the three resulting possible splits for the case where `m==3`: the split index can be either `3`, `4` or `5`: we will choose the option for which the sum of the variances for the two sets is minimal.

In the diagram we show the projection of the points, since it's visually more intuitive; for the same reason, we then rotate the axis and the projections 90° clockwise, remove the points' labels and index the projected points from left to right: in our code, we would have to sort our points according to the y coordinate (as we have seen in listing 10.12), and then we can just consider their indices; an alternative could be using indirect sorting and keeping a table of sorted/unsorted indices, but this would substantially complicate the remaining code.

As noticed, we have 8 points to split: we can deduce that the parameter `M`, the maximum number of leaves/children for a tree node, is equal to 7, and thus `m`, the minimum number of entries, can only be equal to 2 or 3 (technically it could also be 1, but that's a choice that would produce skewed trees, and usually it's not even worth implementing these trees if we use `m==1`).

It's worth reminding that the value for `m` *must* be chosen at the time of creation of our Ss-Tree, and therefore it is fixed when we call `split`: here we are just reasoning about how this choice influences how the splits are performed and ultimately the structure of the tree.